

Contenidos

1	Conceptos sobre subida de ficheros al servidor	1
1.1	Subida de ficheros	1
1.2	El elemento file en un input html	1
1.3	Configurar PHP	2
1.4	Manejo de la subida de ficheros en PHP	3
1.5	Medidas de seguridad	4
1.6	El campo de formulario MAX_FILE_SIZE	7
1.7	Ejemplo de subida de ficheros con PHP	7
1.8	Subida de múltiples ficheros	9

1 Conceptos sobre subida de ficheros al servidor

1.1 Subida de ficheros

Con PHP, es fácil subir archivos al servidor. Sin embargo, con la facilidad viene el peligro, ¡así que cuidado al permitir la carga de archivos!

1.2 El elemento file en un input html

Un elemento `<input>` con `type="file"` permite seleccionar uno o más ficheros y subirlos al servidor a través del envío (submit) de un formulario:

```
<input type="file" id="file" name="file">
```

El elemento value del `<input>` contendrá la ruta al archivo seleccionado. Para cargar varios archivos, agrega el atributo `multiple` al `<input>` elemento de esta manera:

```
<input type="file" id="file" name="file" multiple>
```

En este caso, el value atributo contendrá la ruta del primer archivo en la lista de archivos seleccionados. Tenga en cuenta que aprenderá a cargar varios archivos en otro documento.

Para permitir que se carguen ciertos tipos de archivos, hay que utilizar el atributo `accept`. El valor del atributo `accept` es un especificador de tipo de archivo único, que puede ser:

- Una extensión de nombre de archivo válida que no distingue entre mayúsculas y minúsculas, por ejemplo, `.jpg`, `.pdf`, `.txt`
- Una cadena de tipo MIME válida
- O una cadena como `image/*` (cualquier archivo de imagen), `video/*` (cualquier archivo de vídeo), `audio/*` (cualquier archivo de audio).

Si usamos múltiples especificadores de tipo de fichero, tenemos que separarlos por comas. Ej:

```
<input type="file" accept="image/png, image/jpeg" name="file">
```

El formulario con el `input` para subida de ficheros debe tener el atributo `enctype` con valor `multipart/form-data`, para permitir que el navegador pueda subir ficheros:

```
<form enctype="multipart/form-data" action="index.php" method="post">  
</form>
```

Ejemplo de formulario para subir un fichero:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>PHP File Upload</title>
</head>
<body>
<form enctype="multipart/form-data" action="upload.php" method="post">
    <div>
        <label for="file">Select a file:</label>
        <input type="file" id="file" name="file"/>
    </div>
    <div>
        <button type="submit">Upload</button>
    </div>
</form>
</body>
</html>

```

1.3 Configurar PHP

PHP tiene algunas opciones para controlar la subida de ficheros. Se encuentran en `php.ini`.

Para saber dónde está `php.ini` podemos usar la función de PHP `php_ini_loaded_file()`:

```

<?php

echo php_ini_loaded_file();

```

Las variables de configuración importantes para la subida de ficheros son:

```

; Whether to allow HTTP file uploads.
file_uploads=On

; Temporary directory for HTTP uploaded files (will use system default
; if not specified).
upload_tmp_dir="\tmp" ; o donde

; Maximum allowed size for uploaded files.
upload_max_filesize=2M

; Maximum number of files that can be uploaded via a single request
max_file_uploads=20

```

file_uploads `file_upload` debe estar a «On» para permitir la subida. Por defecto ya lo está.

upload_max_filesize `upload_max_filesize` especifica el tamaño máximo del fichero a subir. Por defecto son 2M (MB). Si tenemos un error que diga que el tamaño del fichero excede a `upload_max_filesize`, hay que incrementar ese valor.

upload_tmp_dir El directorio que, temporalmente va a almacenar el fichero a subir, se guarda en la variable `upload_tmp_dir`.

post_max_size `post_max_size` especifica el tamaño máximo de los datos en una cabecera POST. El tamaño especificado en `post_max_size` debe ser mayor que el de `upload_max_size`, puesto que además del fichero, en la cabecera van otros datos.

max_file_uploads `max_file_uploads` limita el número de ficheros que se pueden subir de una vez.

1.4 Manejo de la subida de ficheros en PHP

Para acceder a la información de un fichero subido, se utiliza el array `$_FILES`. Por ejemplo, si el nombre del elemento de entrada del fichero es `file`, se puede acceder a ese fichero a través de `$_FILES['file']`.

`$_FILE['file']` es un array asociativo con las siguientes claves:

- `name`: es el nombre del fichero subido.
- `type`: es el tipo MIME del archivo cargado, por ejemplo, `image/jpeg` para una imagen JPEG o `application/pdf` para un documento PDF.
- `size`: es el tamaño del fichero en bytes.
- `tmp_name`: es el fichero temporal en el servidor del fichero subido. Si el fichero cargado es demasiado grande, `tmp_name` es `none`.
- `error`: es el código de error que describe el estado de carga, por ejemplo, `UPLOAD_ERR_OK` significa que el archivo se cargó correctamente.

El array asociativo `MESSAGES` contiene los códigos de error que podemos encontrar:

```
<?php
const MESSAGES = [
    UPLOAD_ERR_OK => 'File uploaded successfully',
    UPLOAD_ERR_INI_SIZE => 'File is too big to upload',
    UPLOAD_ERR_FORM_SIZE => 'File is too big to upload',
    UPLOAD_ERR_PARTIAL => 'File was only partially uploaded',
    UPLOAD_ERR_NO_FILE => 'No file was uploaded',
    UPLOAD_ERR_NO_TMP_DIR => 'Missing a temporary folder on the server',
    UPLOAD_ERR_CANT_WRITE => 'File is failed to save to disk.',
    UPLOAD_ERR_EXTENSION => 'File is not allowed to upload to this server',
];
```

Para ver un mensaje basado en un código de error, simplemente puede buscarlo en `MESSAGES` de esta manera:

```
<?php
$message = MESSAGES[$_FILES['file']['error']];
```

Cuando un archivo se carga correctamente, se almacena en un directorio temporal en el servidor. Y puede usar la función `move_uploaded_file()` para mover el archivo del directorio temporal a otro.

La función `move_uploaded_file()` acepta dos argumentos:

- `filename`: es el nombre de fichero del archivo cargado que es `$_FILES['file']['tmp_name']`.
- `destination`: es el destino del archivo movido.

La función `move_uploaded_file()` devuelve `true` si el fichero se mueve con éxito; de lo contrario, devuelve `false`.

1.5 Medidas de seguridad

No se puede confiar en toda la información de la variable `$_FILES`, excepto en `tmp_name`. Los piratas pueden manipular `$_FILES` y cargar scripts maliciosos en el servidor.

Para evitarlo hay que validar la información en `$_FILES`.

Primero, verificamos si el nombre del fichero que se ha introducido en el input, está en la variable `$_FILES` usando `isset()`:

```
<?php
if(! isset($_FILES['file']) ) {
    // error
}
```

En el ejemplo `file` es el nombre del campo input del formulario.

Segundo, comprobar el tamaño del fichero con `filesize()` y comparar el resultado con el máximo permitido. No se debe confiar en el campo `size` de `$_FILE`. Por ejemplo:

```
<?php
const MAX_SIZE = 5 * 1024 * 1024; // 5MB

if (filesize($_FILES['file']['tmp_name']) > MAX_SIZE) {
    // error
}
```

Tenga en cuenta que `MAX_SIZE` no debe ser mayor que `upload_max_filesize`, especificado en el archivo `php.ini`.

El tamaño de un archivo está en bytes, que no es muy legible por humanos. Para hacerlo más legible, podemos definir una función que convierta los bytes a un formato legible, por ejemplo, 1.20M, 2.51G:

```
<?php
function format_filesize(int $bytes, int $decimals = 2): string
{
    $units = 'BKMGTGTP';
    $factor = floor((strlen($bytes) - 1) / 3);

    return sprintf("%.{$decimals}f", $bytes / pow(1024, $factor)) .
        $units[(int)$factor];
}
```

En tercer lugar, validamos el tipo MIME del fichero con los tipos de ficheros permitidos. Para hacerlo, definimos una lista de archivos permitidos:

```
<?php
const ALLOWED_FILES = [
    'image/png' => 'png',
    'image/jpeg' => 'jpg'
];
```

Para obtener el tipo MIME real de un archivo, utiliza tres funciones: `finfo_open()`, `finfo_file()` y `finfo_close()`.

- El `finfo_open()` devuelve un nuevo recurso `fileinfo`.
- El `finfo_file()` devuelve la información sobre el archivo.
- El `finfo_close()` cierra el recurso `fileinfo`.

Para que sea fácil y reutilizable, se puede definir una función `get_mime_type()` como esta:

```
<?php
function get_mime_type(string $filename)
{
    $info = finfo_open(FILEINFO_MIME_TYPE);
    if (!$info) {
        return false;
    }

    $mime_type = finfo_file($info, $filename);
    finfo_close($info);

    return $mime_type;
}
```

La función `get_mime_type()` acepta un nombre de archivo y devuelve el tipo MIME del archivo. Devolverá `false` si se produce un error.

La Autoridad de Números Asignados en Internet (IANA) está a cargo de todos los tipos MIME oficiales, y puede encontrar la lista completa en su [página de tipos MIME](#).

Si ocurre un error o fallo la validación, se puede configurar un mensaje flash y redirigir el navegador a la página de carga. La siguiente función establece un mensaje flash y realiza una redirección:

```
<?php
function redirect_with_message(string $message, string $type=FLASH_ERROR,
    string $name='upload', string $location='index.php'): void
{
    flash($name, $message, $type);
    header("Location: $location", true, 303);
    exit;
}
```

Dado que todas estas funciones `get_mime_type()`, `format_filesize()` y `redirect_with_message()` son reutilizables, las podemos meter en un fichero `functions.php`:

```
<?php
<?php

/**
 * Messages associated with the upload error code
 */
const MESSAGES = [
    UPLOAD_ERR_OK => 'File uploaded successfully',
    UPLOAD_ERR_INI_SIZE => 'File is too big to upload',
    UPLOAD_ERR_FORM_SIZE => 'File is too big to upload',
    UPLOAD_ERR_PARTIAL => 'File was only partially uploaded',
    UPLOAD_ERR_NO_FILE => 'No file was uploaded',
```

```

        UPLOAD_ERR_NO_TMP_DIR => 'Missing a temporary folder on the server',
        UPLOAD_ERR_CANT_WRITE => 'File is failed to save to disk.',
        UPLOAD_ERR_EXTENSION => 'File is not allowed to upload to this server',
    ];

    /**
     * Return a mime type of file or false if an error occurred
     *
     * @param string $filename
     * @return string | bool
     */
    function get_mime_type(string $filename)
    {
        $info = finfo_open(FILEINFO_MIME_TYPE);
        if (!$info) {
            return false;
        }

        $mime_type = finfo_file($info, $filename);
        finfo_close($info);

        return $mime_type;
    }

    /**
     * Return a human-readable file size
     *
     * @param int $bytes
     * @param int $decimals
     * @return string
     */
    function format_filesize(int $bytes, int $decimals = 2): string
    {
        $units = 'BKMGT';
        $factor = floor((strlen($bytes) - 1) / 3);

        return sprintf("%.{$decimals}f", $bytes / pow(1024, $factor)) .
            $units[(int)$factor];
    }

    /**
     * Redirect user with a session based flash message
     * @param string $message
     * @param string $type
     * @param string $name
     * @param string $location
     * @return void
     */

```

```
function redirect_with_message(string $message,
string $type=FLASH_ERROR, string $name='upload',
string $location='index.php'): void
{
    flash($name, $message, $type);
    header("Location: $location", true, 303);
    exit;
}
```

1.6 El campo de formulario MAX_FILE_SIZE

Si ponemos un campo con el nombre MAX_FILE_SIZE antes de un elemento de entrada de archivo en el formulario, PHP usará ese valor en lugar de upload_max_filesize para validar el tamaño del archivo.

Ejemplo:

```
<?php
<form enctype="multipart/form-data" action="upload.php" method="post">
    <div>
        <label for="file">Select a file:</label>
        <input type="hidden" name="MAX_FILE_SIZE" value="10240"/>
        <input type="file" id="file" name="file"/>
    </div>
    <div>
        <button type="submit">Upload</button>
    </div>
</form>
```

En este ejemplo, MAX_FILE_SIZE es 10KB. Si carga un archivo que es más grande que 10KB, PHP generará un error. Sin embargo, es fácil manipular este campo, por lo que nunca debe confiar en él por motivos de seguridad.

A tener en cuenta: no se puede establecer la MAX_FILE_SIZE con un valor más grande que la del upload_max_filesize en php.ini.

1.7 Ejemplo de subida de ficheros con PHP

Primero creamos esta estructura de ficheros:

```
inc
|   flash.php
|   functions.php
index.php
upload.php
uploads
```

Añadimos el siguiente formulario para subir ficheros a index.php:

```
<?php
session_start();
require_once __DIR__ . '/inc/flash.php';
?>
<!DOCTYPE html>
<html lang="en">
```

```

<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <link rel="stylesheet" href="https://www.phptutorial.net/app/css/style.css"/>
    <title>PHP File Upload</title>
</head>
<body>

<?php flash('upload') ?>

<main>
    <form enctype="multipart/form-data" action="upload.php" method="post">
        <div>
            <label for="file">Select a file:</label>
            <input type="file" id="file" name="file"/>
        </div>
        <div>
            <button type="submit">Upload</button>
        </div>
    </form>
</main>
</body>
</html>

```

El fichero `upload.php` se encargará del proceso de subida de ficheros.

En tercer lugar, agregue el siguiente código `upload.php` para procesar el archivo cargado:

```

<?php
//iniciamos sesión
session_start();
//cargamos
require_once __DIR__ . '/inc/flash.php';
require_once __DIR__ . '/inc/functions.php';

const ALLOWED_FILES = [
    'image/png' => 'png',
    'image/jpeg' => 'jpg'
];

const MAX_SIZE = 5 * 1024 * 1024; // 5MB

const UPLOAD_DIR = __DIR__ . '/uploads';

$is_post_request = strtolower($_SERVER['REQUEST_METHOD']) === 'post';
$has_file = isset($_FILES['file']);

if (!$is_post_request || !$has_file) {
    redirect_with_message('Invalid file upload operation', FLASH_ERROR);
}

```



```

//
$status = $_FILES['file']['error'];
$filename = $_FILES['file']['name'];
$tmp = $_FILES['file']['tmp_name'];

// an error occurs
if ($status !== UPLOAD_ERR_OK) {
    redirect_with_message($messages[$status], FLASH_ERROR);
}

// validate the file size
$filesize = filesize($tmp);
if ($filesize > MAX_SIZE) {
    redirect_with_message('Error! your file size is ' .
        format_filesize($filesize) . ' , which is bigger than allowed size ' .
        format_filesize(MAX_SIZE), FLASH_ERROR);
}

// validate the file type
$mime_type = get_mime_type($tmp);
if (!in_array($mime_type, array_keys(ALLOWED_FILES))) {
    redirect_with_message('The file type is not allowed to upload',
        FLASH_ERROR);
}

// set the filename as the basename + extension
$uploaded_file = pathinfo($filename, PATHINFO_FILENAME) . '.'
    . ALLOWED_FILES[$mime_type];

// new file location
$filepath = UPLOAD_DIR . '/' . $uploaded_file;

// move the file to the upload dir
$success = move_uploaded_file($tmp, $filepath);
if ($success) {
    redirect_with_message('The file was uploaded successfully.',
        FLASH_SUCCESS);
}

redirect_with_message('Error moving the file to the upload folder.',
    FLASH_ERROR);

```

1.8 Subida de múltiples ficheros

index.php:

```

<?php
session_start();
require_once __DIR__ . '/inc/flash.php';
?>

```

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>PHP upload multiple files</title>
    <link rel="stylesheet"
        href="https://www.phptutorial.net/app/css/style.css" />
</head>
<body>
<?php flash('upload') ?>
<main>
    <form action="upload.php" method="post" enctype="multipart/form-data">
        <div>
            <label for="files">Select files to upload:</label>
            <input type="file" name="files[]" id="files" multiple required/>
        </div>
        <div>
            <button type="submit">Upload</button>
        </div>
    </form>
</main>
</body>
</html>

```

upload.php

```

<?php

session_start();

require_once __DIR__ . '/inc/flash.php';
require_once __DIR__ . '/inc/functions.php';

const ALLOWED_FILES = [
    'image/png' => 'png',
    'image/jpeg' => 'jpg'
];

const MAX_SIZE = 5 * 1024 * 1024; // 5MB

const UPLOAD_DIR = __DIR__ . '/uploads';

$is_post_request = strtolower($_SERVER['REQUEST_METHOD']) === 'post';
$has_files = isset($_FILES['files']);

if (!$is_post_request || !$has_files) {
    redirect_with_message('Invalid file upload operation', FLASH_ERROR);
}

```

```

$files = $_FILES['files'];
$file_count = count($files['name']);

// validation
$errors = [];
for ($i = 0; $i < $file_count; $i++) {
    // get the uploaded file info
    $status = $files['error'][$i];
    $filename = $files['name'][$i];
    $tmp = $files['tmp_name'][$i];

    // an error occurs
    if ($status !== UPLOAD_ERR_OK) {
        $errors[$filename] = MESSAGES[$status];
        continue;
    }
    // validate the file size
    $filesize = filesize($tmp);

    if ($filesize > MAX_SIZE) {
        // construct an error message
        $message =
            sprintf("The file %s is %s which is greater than the allowed size %s",
                $filename,
                format_filesize($filesize),
                format_filesize(MAX_SIZE));

        $errors[$filename] = $message;
        continue;
    }

    // validate the file type
    if (!in_array(get_mime_type($tmp), array_keys(ALLOWED_FILES))) {
        $errors[$filename] = "The file $filename is allowed to upload";
    }
}

if ($errors) {
    redirect_with_message(
        format_messages('The following errors occurred:', $errors), FLASH_ERROR);
}

// move the files
for ($i = 0; $i < $file_count; $i++) {
    $filename = $files['name'][$i];
    $tmp = $files['tmp_name'][$i];
    $mime_type = get_mime_type($tmp);

```

```

// set the filename as the basename + extension
$uploaded_file = pathinfo($filename, PATHINFO_FILENAME) . '.' .
    ALLOWED_FILES[$mime_type];

// new filepath
$filepath = UPLOAD_DIR . '/' . $uploaded_file;

// move the file to the upload dir
$success = move_uploaded_file($tmp, $filepath);
if(!$success) {
    $errors[$filename] = "The file $filename was failed to move.";
}
}
//en 3 líneas
$errors ?
    redirect_with_message(
format_messages(
    'The following errors occurred:', $errors), FLASH_ERROR) :
    redirect_with_message(
    'All the files were uploaded successfully.', FLASH_SUCCESS);

```

<https://blog.filestack.com/thoughts-and-knowledge/php-file-upload/>

Ejemplo con base de datos y msqli, pero fácilmente trasladable a PDO

Subir directorios completos: https://php.watch/versions/8.1/protect-T1-textdollar_FILES-full-